

37. Allumer la couleur : rouge, vert, bleu

Sur Game Boy Color, chaque indice 0–3 d'une tuile devient une vraie couleur, que tu choisis.

1. Ce qu'il faut comprendre

La Game Boy Color garde **exactement les mêmes tuiles** : deux bits par pixel, des indices de 0 à 3. Ce qui change, c'est la palette. Au lieu de dire « indice 2 → nuance 2 », elle dit « indice 2 → **ce rouge-là** ».

`couleurFond(palette, teinte, rouge, vert, bleu)` règle **une** teinte : la palette (0 à 7, on en parle plus loin), l'indice concerné (0 à 3), puis trois **composantes** de **0 à 31**. 0 : la lumière de cette couleur est éteinte ; 31 : elle est à fond.

Cinq bits par composante : $32 \times 32 \times 32 = \mathbf{32\ 768\ couleurs}$ au choix. `31, 31, 31` est le blanc, `0, 0, 0` le noir, `31, 0, 0` le rouge pur.

Le texte est dessiné en indice 3 sur un fond d'indice 0 : ici, il prend donc le bleu nuit de la teinte 3 sur le blanc de la teinte 0.

Dans l'atelier, la liste en haut à gauche choisit la console, l'une **ou** l'autre : « **En couleur** » (une cartouche `.gbc` pour la Game Boy Color) ou « **Game Boy** » (un `.gb`, les quatre nuances, rien de plus). En « Game Boy », `couleurFond()` est **refusé** — une Game Boy d'origine n'a pas de registre de couleur, et le compilateur préfère le dire.

À toi : change les quatre teintes pour faire un dégradé du jaune au rouge.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **488 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
main()         118 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← ecran() – attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte – attend de pouvoir écrire une case
AdresseCase    10 o ← poser() – trouve l'adresse d'une case de l'écran
EcrireTexte    10 o ← texte() – écrit un texte, lettre par lettre
CopierTuiles   85 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles         128 o ← CopierTuiles – 7 lettres sur 43, et 1 tuile dessinée
"EN COULEUR"   10 o ← texte() – un texte gravé
```

Lettres gravées : CELNORU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
couleurFond() × 4    48 o sur place, dans main()
ecran()       × 2    21 o sur place, dans main() + AttendreVBlank (16 o)
poser()       × 1    13 o sur place, dans main() + AdresseCase (10 o)
```

```

texte()      × 1    11 o sur place, dans main() + EcrireTexte (10 o)
image()      × 1    3 o sur place, dans main() + AttendreImage (28 o)

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME (Game Boy Color) : la même bande de 4 indices qu'en nuances,
// mais chaque indice reçoit une vraie couleur : blanc, orange, prune, bleu nuit.
//
// Ce qui est nouveau : couleurFond(palette, indice, rouge, vert, bleu)
// palette : laquelle des 8 palettes du fond (0 à 7) ; ici la 0.
// indice : lequel des 4 indices de la tuile on colore (0 à 3).
// rouge, vert, bleu : trois lumières, de 0 (éteinte) à 31 (à fond).
// Exemples : 31, 31, 31 = blanc ; 0, 0, 0 = noir ; 31, 0, 0 = rouge pur.
// Attention : il faut choisir la console « En couleur » dans l'atelier.

#include <Tuile>           // un dessin de 8 × 8 pixels
#include <couleurFond>     // choisit une couleur d'une palette du fond
#include <ecran>           // éteint ou rallume l'écran
#include <texte>           // écrit un texte à l'écran
#include <poser>           // pose une tuile sur une case du fond

```

2. UN DESSIN — BANDES

```

Tuile BANDES = {           // 2 rangées de chaque indice : 0, 1, 2, 3.
    "00000000", "00000000", "11111111", "11111111",
    "22222222", "22222222", "33333333", "33333333",
};

```



à l'écran

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

int main() {               // Le jeu commence ici.
    // La palette 0 : quatre teintes, de 0 à 31 pour rouge, vert, bleu.
    couleurFond(0, 0, 31, 31, 31); // indice 0 : blanc (les trois lumières à fond)
    couleurFond(0, 1, 31, 20, 0);  // indice 1 : orange (beaucoup de rouge, un peu de vert)
    couleurFond(0, 2, 20, 0, 10);  // indice 2 : prune (rouge et un peu de bleu)
    couleurFond(0, 3, 0, 0, 12);   // indice 3 : bleu nuit (un peu de bleu seulement)

    ecran(0);               // Dessin écran éteint.
    texte(2, 2, "EN COULEUR"); // Les lettres sont en indice 3 sur un fond d'indice 0 :
                                // du bleu nuit sur du blanc.

    for (uint8_t c = 0; c < 20; c++) {
        poser(c, 7, BANDES); // La bande, sur toute la ligne 7.
    }
    ecran(1);               // On rallume.

    while (true) {          // La boucle du jeu.
        image();
    }
}

```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Quatre bandes : blanc, orange, prune, bleu nuit — et le texte en bleu nuit sur blanc.

5. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la console est passée en couleur
- ✓ chaque indice a pris sa couleur (31/31/31 31/20/0 20/0/10 0/0/12)
- ✓ le texte prend la teinte 3

← [36. Faire clignoter un personnage touché](#)

[sommaire](#)

[38. Mélanger la lumière](#) →